

■ LEVEL 3 • TWO POINTERS

10 Problems • Pattern Recognition • Optimal C++ • Visible Indentation

Reference pattern: Two positions move through the data while eliminating many candidates, often with sorted data or opposite ends. The key question is not simply “use two pointers”, but **why one pointer can safely move**.

Core practice from the reference handbook: Valid Palindrome, Two Sum II, 3Sum, Container With Most Water, Remove Duplicates from Sorted Array, Trapping Rain Water.

Expanded: Four additional problems strengthen read/write pointers, opposite-end comparisons, sorting + two pointers, and reverse scanning.

LEVEL 3 • PROBLEM INDEX

#	Problem	Pattern	LeetCode
1	Valid Palindrome	Two Pointers	#125
2	Two Sum II - Input Array Is Sorted	Two Pointers / Sorted Array	#167
3	3Sum	Sorting + Two Pointers	#15
4	Container With Most Water	Two Pointers / Greedy Movement	#11
5	Remove Duplicates from Sorted Array	Two Pointers / In-Place	#26
6	Trapping Rain Water	Two Pointers / Boundary State	#42
7	Move Zeroes	Two Pointers / Read-Write	#283
8	Squares of a Sorted Array	Two Pointers / Opposite Ends	#977
9	Boats to Save People	Sorting + Two Pointers	#881
10	Backspace String Compare	Two Pointers / Reverse Scan	#844

Recognition checklist: Is the data sorted? Are we scanning from opposite ends? Is there a read/write boundary? Can one pointer move without losing a valid candidate? Can sorting expose the property?

PROBLEM 1 Valid Palindrome

■ Pattern: Two Pointers

LeetCode: #125

■ Input	A string s containing letters, digits, and other characters.
■ Output	Return true if s is a palindrome after converting letters to lowercase and removing non-alphanumeric characters.
■ Constraints	$1 \leq s.length \leq 2 * 10^5$.
■ Main Idea / Pattern	Put one pointer at each end. Ignore non-alphanumeric characters and compare the normalized characters. Move inward only after a valid comparison.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
class Solution {
public:
    bool isPalindrome(string s) {
        int left = 0;
        int right = s.size() - 1;
        while (left < right) {
            while (left < right && !isalnum(s[left])) {
                left++;
            }
            while (left < right && !isalnum(s[right])) {
                right--;
            }
            if (tolower(s[left]) != tolower(s[right])) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 2 Two Sum II - Input Array Is Sorted

■ Pattern: Two Pointers / Sorted Array

LeetCode: #167

■ Input	A 1-indexed array of integers sorted in non-decreasing order and a target.
■ Output	Return the two indices whose values sum to target.
■ Constraints	$2 \leq \text{numbers.length} \leq 3 \cdot 10^4$.
■ Main Idea / Pattern	If the sum is too small, increase left. If it is too large, decrease right. Sorted order makes each movement safe.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int left = 0;
        int right = numbers.size() - 1;
        while (left < right) {
            int sum = numbers[left] + numbers[right];
            if (sum == target) {
                return {left + 1, right + 1};
            }
            if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
        return {};
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 3 3Sum

■ Pattern: Sorting + Two Pointers

LeetCode: #15

■ Input	An integer array nums.
■ Output	Return all unique triplets whose sum is zero.
■ Constraints	3 <= nums.length <= 3000.
■ Main Idea / Pattern	Sort first. Fix one value, then use left/right pointers to find the remaining pair. Skip duplicates at every level.
■ Time Complexity	$O(n^2)$
■ Space Complexity	$O(1)$ extra, excluding output

■ Optimal C++ Code

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        vector<vector<int>> ans;

        for (int i = 0; i < nums.size(); i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            int left = i + 1;
            int right = nums.size() - 1;
            while (left < right) {
                long long sum = (long long)nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    ans.push_back({nums[i], nums[left], nums[right]});
                    while (left < right && nums[left] == nums[left + 1]) {
                        left++;
                    }
                    while (left < right && nums[right] == nums[right - 1]) {
                        right--;
                    }
                    left++;
                    right--;
                } else if (sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return ans;
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 4 Container With Most Water

■ Pattern: Two Pointers / Greedy Movement

LeetCode: #11

■ Input	An integer array height representing vertical lines.
■ Output	Return the maximum amount of water a container can store.
■ Constraints	$2 \leq n \leq 10^5$; heights are non-negative.
■ Main Idea / Pattern	Area is limited by the shorter line. Move the pointer at the shorter line because keeping it while reducing width cannot improve the area.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int left = 0;
        int right = height.size() - 1;
        int best = 0;
        while (left < right) {
            int width = right - left;
            int h = min(height[left], height[right]);
            best = max(best, width * h);
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }
        return best;
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 5 Remove Duplicates from Sorted Array

■ Pattern: Two Pointers / In-Place

LeetCode: #26

■ Input	A sorted integer array nums.
■ Output	Remove duplicates in-place so each unique value appears once and return the number of unique values.
■ Constraints	$1 \leq \text{nums.length} \leq 3 * 10^4$.
■ Main Idea / Pattern	Keep a write pointer for the next unique position. Scan with a read pointer and write only when the value changes.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
class Solution {
public:
    int removeDuplicates(vector<int>& nums) {
        int write = 1;
        for (int read = 1; read < nums.size(); read++) {
            if (nums[read] != nums[read - 1]) {
                nums[write] = nums[read];
                write++;
            }
        }
        return write;
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 6 Trapping Rain Water

■ Pattern: Two Pointers / Boundary State

LeetCode: #42

■ Input	An array height representing bar heights.
■ Output	Return the total amount of trapped rain water.
■ Constraints	$1 \leq \text{height.length} \leq 2 * 10^4$.
■ Main Idea / Pattern	Keep leftMax and rightMax. The side with the smaller boundary determines how much water can be finalized at that side.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
class Solution {
public:
    int trap(vector<int>& height) {
        int left = 0;
        int right = height.size() - 1;
        int leftMax = 0;
        int rightMax = 0;
        int water = 0;
        while (left < right) {
            if (height[left] <= height[right]) {
                leftMax = max(leftMax, height[left]);
                water += leftMax - height[left];
                left++;
            } else {
                rightMax = max(rightMax, height[right]);
                water += rightMax - height[right];
                right--;
            }
        }
        return water;
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 7 Move Zeroes

■ **Pattern:** Two Pointers / Read-Write

LeetCode: #283

■ Input	An integer array nums.
■ Output	Move all zeroes to the end while maintaining the relative order of non-zero elements.
■ Constraints	$1 \leq \text{nums.length} \leq 10^4$.
■ Main Idea / Pattern	Use a write pointer for the next non-zero position and a read pointer to scan the array. Fill the remaining positions with zero.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int write = 0;
        for (int read = 0; read < nums.size(); read++) {
            if (nums[read] != 0) {
                swap(nums[write], nums[read]);
                write++;
            }
        }
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 8 Squares of a Sorted Array

■ Pattern: Two Pointers / Opposite Ends

LeetCode: #977

■ Input	A non-decreasing sorted integer array nums.
■ Output	Return an array of the squares sorted in non-decreasing order.
■ Constraints	1 <= nums.length <= 10 ⁴ .
■ Main Idea / Pattern	The largest square comes from either the most negative left value or the largest right value. Compare absolute values and fill the result from the end.
■ Time Complexity	O(n)
■ Space Complexity	O(n)

■ Optimal C++ Code

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& nums) {
        int n = nums.size();
        vector<int> ans(n);
        int left = 0;
        int right = n - 1;
        for (int i = n - 1; i >= 0; i--) {
            if (abs(nums[left]) > abs(nums[right])) {
                ans[i] = nums[left] * nums[left];
                left++;
            } else {
                ans[i] = nums[right] * nums[right];
                right--;
            }
        }
        return ans;
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 9 Boats to Save People

■ Pattern: Sorting + Two Pointers

LeetCode: #881

■ Input	An array people of weights and a boat weight limit.
■ Output	Return the minimum number of boats needed when each boat carries at most two people.
■ Constraints	$1 \leq \text{people.length} \leq 5 * 10^4$.
■ Main Idea / Pattern	Sort weights. Pair the lightest with the heaviest when they fit; otherwise the heaviest must ride alone.
■ Time Complexity	$O(n \log n)$
■ Space Complexity	$O(1)$ extra

■ Optimal C++ Code

```
class Solution {
public:
    int numRescueBoats(vector<int>& people, int limit) {
        sort(people.begin(), people.end());

        int left = 0;
        int right = people.size() - 1;
        int boats = 0;

        while (left <= right) {
            if (people[left] + people[right] <= limit) {
                left++;
            }
            right--;
            boats++;
        }

        return boats;
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.

PROBLEM 10 Backspace String Compare

■ Pattern: Two Pointers / Reverse Scan

LeetCode: #844

■ Input	Two strings s and t where # represents a backspace.
■ Output	Return true if the two strings are equal after applying backspaces.
■ Constraints	1 <= s.length, t.length <= 200.
■ Main Idea / Pattern	Scan both strings from right to left. A skip counter tells how many characters must be ignored, avoiding construction of the final strings.
■ Time Complexity	O(n + m)
■ Space Complexity	O(1)

■ Optimal C++ Code

```
class Solution {
    char nextChar(string& s, int& i, int& skip) {
        while (i >= 0) {
            if (s[i] == '#') {
                skip++;
                i--;
            } else if (skip > 0) {
                skip--;
                i--;
            } else {
                return s[i];
            }
        }
        return '\0';
    }
public:
    bool backspaceCompare(string s, string t) {
        int i = s.size() - 1;
        int j = t.size() - 1;
        int skipS = 0;
        int skipT = 0;
        while (i >= 0 || j >= 0) {
            char a = nextChar(s, i, skipS);
            char b = nextChar(t, j, skipT);
            if (a != b) return false;
            if (a == '\0' && b == '\0') return true;
            i--;
            j--;
        }
        return true;
    }
};
```

Practice: Dry-run the pointer movement on a small example. Then explain exactly why moving that pointer cannot discard a needed answer.